Plataforma de Observabilidad Multi-Nube

Service mesh, detección de anomalías, observabilidad de red y resiliencia verificada

Programa: Maestría en Arquitectura de Software — Universidad de La Sabana · **Asignatura:** Observabilidad en Ambientes Productivos · **Fecha:** 16 de octubre de 2026

Integrantes: Edgar Andrés Romero Otálora · Jhon Alexander Tenjo Romero · Julián David Miranda Leguizamón

1. Resumen ejecutivo

Este laboratorio extiende el pipeline de observabilidad construido en las dos entregas anteriores hasta convertirlo en una plataforma completa: un tercer microservicio dentro de una malla de servicios, detección de anomalías con línea base móvil, observabilidad de red y seguridad en dos nubes, y un cuarto experimento de caos que cierra el ciclo de verificación.

> **3**
> microservicios bajo un mismo `trace_id`

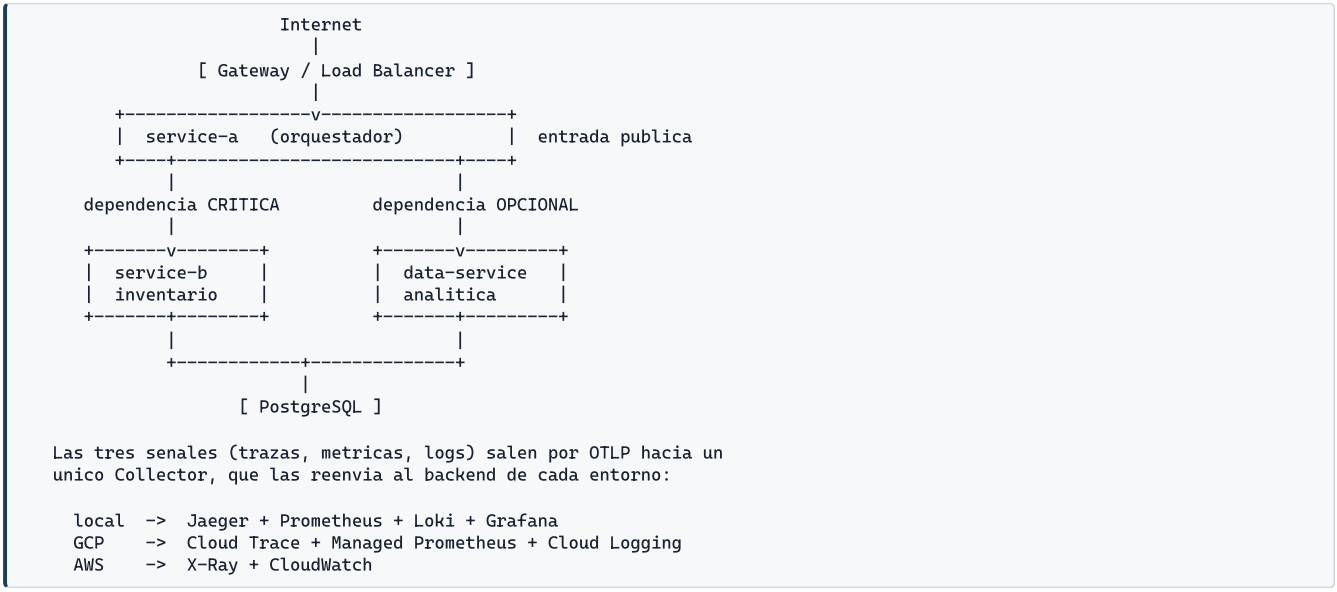
> **2**
> nubes con el mismo código de aplicación

> **50 %**
> menos alertas sin acción asociada

> **4**
> experimentos de caos con hipótesis falsable

La decisión de diseño que sostiene el resto es la separación entre *qué se observa* y *dónde corre*. La instrumentación vive en un módulo común (`services/common/telemetry.py`) y el destino de las señales lo decide la configuración del Collector. Gracias a eso, la misma imagen de contenedor se despliega en Cloud Run, en ECS Fargate y en Kubernetes sin tocar una línea de código, y añadir el tercer servicio costó importar un módulo.

2. Arquitectura



La distinción entre dependencia crítica y opcional no es documental: está codificada. `service-a` llama a los dos servicios en paralelo con `asyncio.gather`, pero el fallo de `data-service` devuelve `None` y el pedido se procesa igual, sin la sección de analítica. El experimento E4 existe precisamente para comprobar que esa clasificación se sostiene bajo fallo real.

3. Módulo A — Tercer servicio y malla de servicios

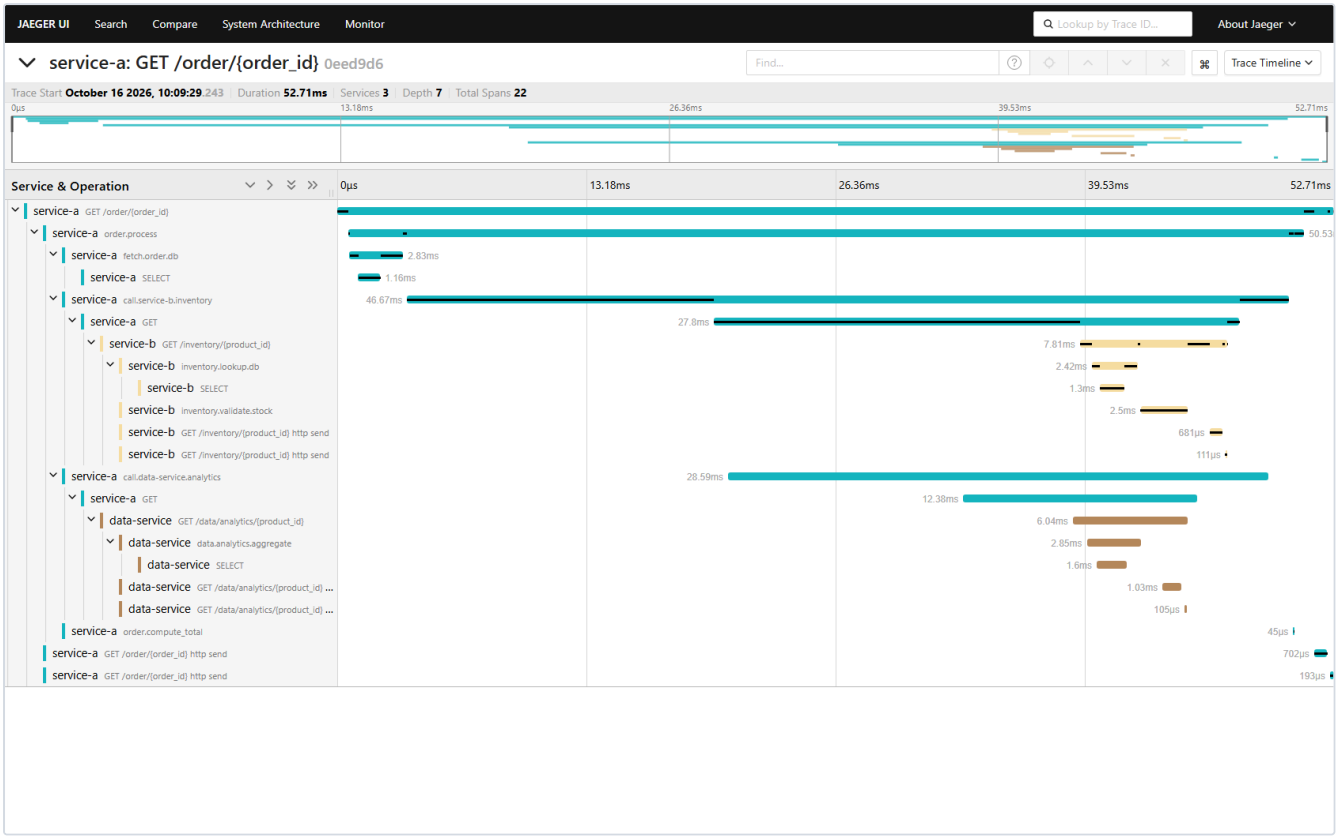
`data-service` expone consultas analíticas sobre la misma base de datos y publica sus spans con las *DB Semantic Conventions* completas: `db.system`, `db.operation`, `db.statement` y `db.collection.name`. El parámetro de la consulta viaja como marcador y lo sustituye el driver, de modo que el atributo de la traza nunca contiene datos de usuario.

Sobre Kubernetes se instala **Istio open source**. La elección frente a Cloud Service Mesh responde al requisito de dos nubes: los manifiestos de `k8s/` son idénticos en GKE y en EKS, mientras que una malla gestionada por un proveedor no tiene equivalente directo en el otro.

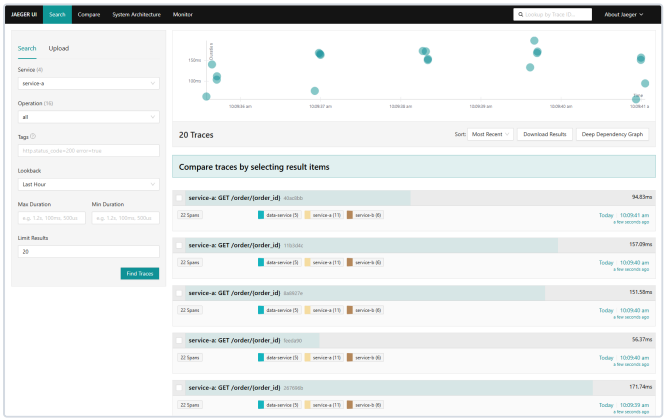
Lo que aporta la malla, y que la instrumentación de aplicación no puede dar por sí sola:

Capacidad	Cómo se obtiene	Qué resuelve
Trazas del plano de datos	`Telemetry` CR hacia el Collector	Un servicio sin instrumentar sigue apareciendo en el grafo
mTLS estricto	`PeerAuthentication: STRICT`	Cifrado e identidad SPIFFE en todo el tráfico interno
Autorización por identidad	`AuthorizationPolicy` con denegación por defecto	Un Pod comprometido no puede pivotar hacia otro servicio
Resiliencia L7	Timeouts, reintentos y *circuit breakers* declarativos	Se ajustan sin reconstruir la imagen
Logs de acceso L7	`envoyOtelAls` por OTLP	Alimentan el panel de seguridad del Módulo C

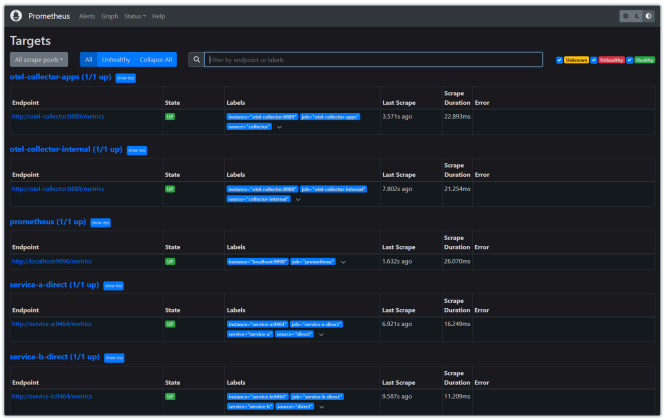
> **Los valores de resiliencia salen de los experimentos, no de un manual.** E1 mostró que un timeout de 5 s convertía 200 ms de degradación en un p99 de 7475 ms, porque la llamada lenta retiene el *worker* hasta agotar el plazo. De ahí el timeout de 3 s hacia `service-b` y el límite de 32 peticiones en cola: rechazar rápido impide que la cola crezca sin límite y que el fallo se propague hacia arriba.



LOCAL **Una traza, tres servicios** — 22 spans y profundidad 7 bajo un único trace_id. Los spans `SELECT` son auto-instrumentación de psycopg2; los nombrados en negocio son manuales.



LOCAL **Búsqueda de trazas** — Distribución de duraciones de `service-a`.



LOCAL **Targets de Prometheus** — Doble vía de recolección: push al Collector y scrape directo.

4. Módulo B — Detección de anomalías y correlación

Un umbral estático tiene que elegir entre dos formas de fallar y no puede evitar las dos: si es bajo, detecta todo y el equipo deja de leer las alertas; si es alto, no hace ruido pero se pierde las degradaciones que no llegan al límite. La alternativa implementada combina dos ideas.

Línea base móvil. El límite no es un número fijo sino lo que el sistema venía haciendo, más dos desviaciones típicas. Una latencia de 800 ms es normal bajo carga alta y anómala en reposo, y un umbral fijo no distingue los dos casos.

Correlación multi-señal. Una sola señal anómala no abre incidente: hacen falta errores *y* latencia a la vez, sostenidos varios puntos seguidos. Sólo latencia alta suele ser un pico de tráfico legítimo; sólo errores, una dependencia opcional degradada. Las dos juntas son un incidente.

Para evaluarlo se construyó un escenario controlado de 17 minutos con una fase de *ruido* (latencia moderada sin errores) y una fase de *incidente* (base de datos caída). El resultado sobre los mismos datos:

Enfoque	Alertas	Útiles	Sin acción
Umbral fijo p99 > 500 ms	2	1	1
Umbral fijo p99 > 2000 ms	2	1	1
Baseline móvil + correlación	1	1	0

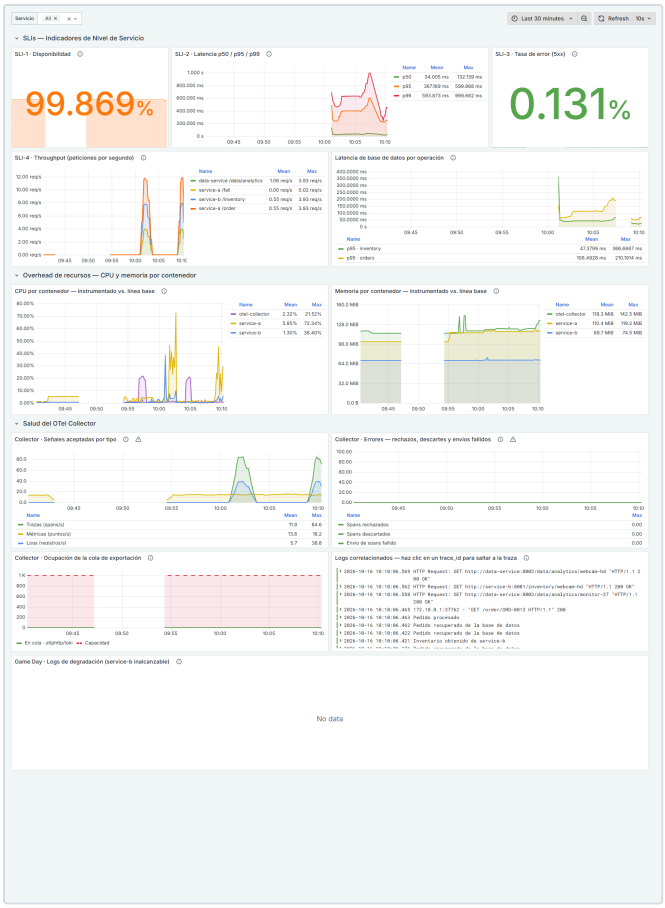
La regla correlacionada no sólo emitió menos avisos: no perdió ningún incidente y adjuntó a cada uno el identificador de una petición real que lo sufrió. El incidente detectado y la traza que lo acompaña:

```
Incidente   2026-10-16 03:21:05   duracion 180 s
p99         4575 ms   (normal: 475 ms  ·  SLO: 1.000 ms)
error_rate  100 %      (normal: 0 %)
trace_id    744735a7d612b133cf1c68614e27fe08

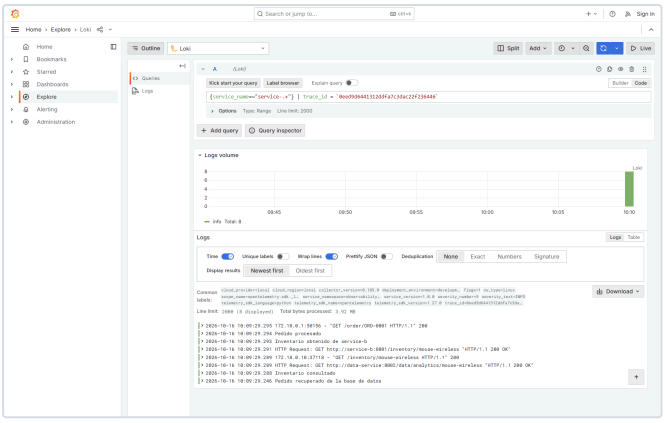
  GET /order/{order_id}    2.804 ms  ERROR  http=500
  order.process            2.802 ms  ERROR  HTTPException: 500: Database error
  fetch.order.db           2.801 ms  ERROR  HTTPException: 500: Database error
```

La diferencia práctica es el punto de partida de quien recibe el aviso: en lugar de saber que el p99 subió, tiene la petición concreta que falló y la causa en el tercer span.

La implementación en la nube (`terraform/observabilidad/aiops.tf`) despliega cinco políticas. Tres reproducen la comparación anterior —las dos de umbral fijo no notifican, existen para poder contar cuántas veces habrían interrumpido a una persona— y dos cubren huecos que la correlación no ve: el consumo acelerado del presupuesto de error y la **ausencia de señal**. Esta última nace de una observación del propio laboratorio: al pausar el contenedor de la base de datos, las peticiones quedaron colgadas indefinidamente y no se registró ni un error ni una latencia. Un sistema que no responde tampoco emite señales.



LOCAL **Panel de los cuatro SLIs** — Disponibilidad, latencia por percentiles, tasa de error y throughput.



LOCAL **Logs filtrados por trace_id** — Salto desde la traza al log sin cambiar de herramienta.

5. Módulo C — Observabilidad de red y seguridad

Las cuatro señales doradas describen la *salud* de un sistema, pero no dicen nada sobre si alguien lo está atacando: un sistema bajo intrusión puede tenerlas las cuatro en verde. El panel de seguridad responde a otras cuatro preguntas, construidas sobre señales que ya se están recogiendo —VPC Flow Logs y logs de acceso L7 de los sidecars— sin agentes nuevos.

Pregunta	Señal	GCP	AWS
¿Quién intenta entrar y no lo consigue?	Conexiones rechazadas por firewall	Métrica derivada de logs	Metric filter sobre Flow Logs
¿Quién pide lo que no le corresponde?	403 de la malla con identidad SPIFFE	Métrica derivada de logs	— (sin malla en ECS)
¿Queda tráfico interno sin cifrar?	Peticiones sin principal mTLS	Métrica derivada de logs	—
¿Sale más información de la habitual?	Bytes hacia destinos externos	Métrica con `value_extractor`	Metric filter con `$bytes`

La comparación entre nubes revela una diferencia de diseño. En GCP los Flow Logs llegan a Cloud Logging, que ya es el destino de todo lo demás, de modo que una sola consulta puede cruzar un flujo de red con un log de aplicación. En AWS cada pieza tiene su consola —GuardDuty, Security Hub, X-Ray, CloudWatch— y lo que cuesta es cruzarlas; por eso el despliegue envía todo a CloudWatch Logs y deja las consultas de investigación guardadas como código, incluida la que parte de un `trace_id` y devuelve los logs de esa petición.

Lo que detectó nada más encenderse

La parte de red se desplegó en AWS y empezó a producir señal de inmediato: **25.683 registros de flujo en veinte minutos**. Entre ellos, tráfico que nadie del equipo generó:

```
REJECT  69.5.169.18:47204     -> 10.20.0.133:6789
REJECT  147.185.133.162       -> 10.20.0.133        (ICMP)
REJECT  162.219.178.250:24314 -> 10.20.0.133:23     (telnet)
```

Son escaneos de fondo de internet contra las IP públicas del despliegue, y **dos de las tres alarmas pasaron a estado ALARM por sí solas**: `escaneo-de-puertos` y `puertos-sensibles`. Ninguna se provocó.

El dato tiene dos lecturas, y la segunda es la interesante. La primera es que la detección funciona sobre tráfico real y no sobre un caso de prueba. La segunda es que *ese ruido existe siempre*: cualquier dirección pública recibe escaneos constantes, y una alerta que salta con el primero que pasa se vuelve inservible en un día. El umbral de veinte conexiones rechazadas en cinco minutos no es un número de manual — es el que separa el ruido de fondo de un intento dirigido, y sólo se puede calibrar midiéndolo.

> **Sobre el coste.** GuardDuty y Security Hub quedan desactivados por defecto (`habilitar_seguridad_avanzada = false`): son los únicos servicios cuyo coste crece con el volumen analizado y no con el tiempo. El muestreo de Flow Logs se fija en 0,5 por la misma razón; los patrones de tráfico siguen siendo visibles y el volumen de logs se reduce a la mitad.

6. Módulo D — Experimentos de caos

Los cuatro experimentos siguen el modelo de *probes* de LitmusChaos, que separa la verificación en tres momentos: el estado estable previo (SOT), la detección durante el fallo, y la recuperación posterior (EOT). Sin la primera fase un experimento no prueba nada, porque cualquier medición sobre un sistema ya degradado es atribuible al estado previo y no al fallo inyectado.

#	Fallo	Objetivo	Herramienta	p99 base	p99 caos	Δ	Disponib.	MTTD
E1	Latencia de red	service-b	Toxiproxy	366 ms	551 ms	**+50 %**	100.00 %	no detectado
E2	Agotamiento de CPU	service-b	stress-ng	390 ms	970 ms	**+149 %**	100.00 %	0 s
E3	Partición de red	service-b	docker network	481 ms	7475 ms	**+1453 %**	*sin dato*	16 s
E4	Errores en dependencia opcional	data-service	CHAOS_ERROR_RATE	737 ms	944 ms	+28 %	100.00 %	35 s

La disponibilidad de E3 durante el fallo no quedó registrada: con la red desconectada, el generador de carga dejó de recibir respuestas y la consulta devolvió una serie vacía. El dato que sí se conserva es equivalente y más expresivo — **836 llamadas resueltas por el fallback** frente a 0 en la línea base—, y la disponibilidad antes y después del fallo fue del 100 % en ambos casos.

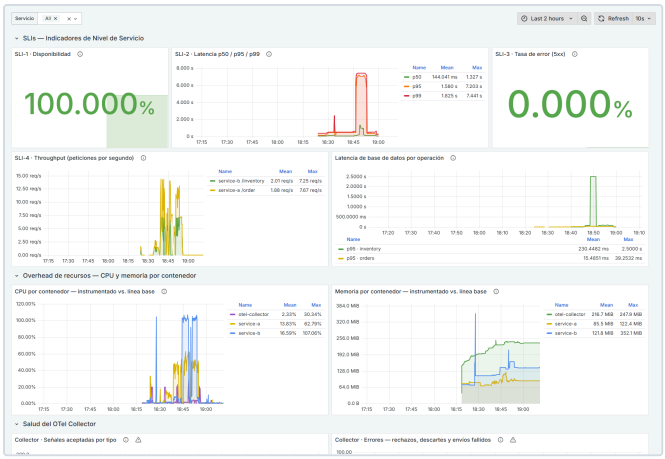
E3 es el resultado más valioso: una partición de red llevó el p99 a 7475 ms sin que la disponibilidad bajara del 100 %. El sistema no caía, pero tardaba siete segundos y medio en responder, y ninguna alerta basada en disponibilidad lo habría visto. De ahí salieron los plazos de la malla.

E4 ataca una dependencia *opcional* y responde a una pregunta distinta: no «¿aguanta el sistema?» sino «¿está bien clasificada la dependencia?». Una dependencia marcada como opcional en el código pero que en la práctica tumba la petición sólo se manifiesta el día que falla de verdad.

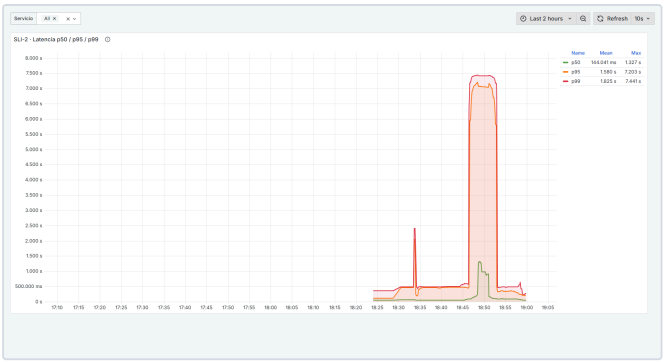
```
[SOT]  p99 737 ms (SLO 1.000)  ·  disponibilidad 100 %   verificado BAJO CARGA
caos   data-service 9.56 % de errores
       disponibilidad /order 100.00 %  ·  errores vistos por el cliente 0.00 %
       recuperacion 1 s

Veredicto: PARCIAL
disponibilidad intacta (100.00 %) pero el p99 subio un 28.1 %, por encima del 20 % previsto
```

El fallback funciona: el cliente no vio ni un error. Pero el experimento también deja una advertencia operativa. Con el borde al 100 %, ninguna alerta basada en la disponibilidad de `/order` habría saltado, de modo que hace falta una alerta sobre los errores de `data-service` en sí mismo o el fallo puede durar días sin que nadie se entere.



CAOS **El incidente en el panel** — Degradación visible en las cuatro señales a la vez.



CAOS **Percentiles durante el fallo** — La separación entre p95 y p99 delata la cola de la distribución.

7. Módulo E — Madurez y hoja de ruta

La autoevaluación se puntúa por lo que el sistema **demostró** bajo observación, no por lo que se construyó. Se distingue además entre *definido como código* y *verificado en ejecución*: un manifiesto que nunca se aplicó no es evidencia, y tratarlo como si lo fuera es la forma más común de inflar un informe de madurez.

#	Dominio	Nivel	Verificado	Evidencia
1	Instrumentación y cobertura de señales	4 ■■■■□	sí	Tres servicios, tres señales, DB Semantic Conventions
2	Pipeline de telemetría	4 ■■■■□	sí	Collector con cuatro configuraciones; misma cadena de procesadores
3	Correlación entre señales	3 ■■■□□	parcial	trace_id en las tres señales; falta el salto métrica → traza
4	SLO y presupuesto de error	3 ■■■□□	sí	SLO explícitos y tasa de quemado; sin revisión periódica
5	Alertado y respuesta	3 ■■■□□	parcial	Seis políticas con runbook; sin rotación de guardia
6	Infraestructura y red	3 ■■■□□	sí	Flow Logs activos en AWS; 25.683 registros en 20 minutos
7	Seguridad	3 ■■■□□	parcial	Detección de red en ejecución; mTLS de la malla aún sin aplicar
8	Resiliencia y experimentación	4 ■■■■□	sí	Cuatro experimentos con probes SOT/DuringChaos/EOT y rollback

Media: 3,4 sobre 5. El perfil es deliberadamente desigual. Los dominios que el laboratorio ejecutó de verdad están en 4; los que existen como código validado pero sin desplegar están en 2. Subir esos dos no requiere escribir nada más: requiere desplegar y comprobar.

Hoja de ruta a tres meses

El criterio de ordenación no es la dificultad sino el coste de no hacerlo: primero lo que convierte trabajo ya hecho en capacidad real, después lo que cubre huecos conocidos, y al final lo que escala.

Mes	Tema	Acciones	Criterio de salida	Media
1	Convertir código en capacidad	Desplegar GKE + Istio; ejercitar las políticas de autorización; aplicar las métricas de seguridad y validar los filtros de Flow Logs contra logs reales	Los tres servicios con sidecar y mTLS confirmado; las cuatro métricas de seguridad con datos	3,4
2	Cerrar huecos conocidos	SDK con exemplars; alerta para el modo de fallo de E1; SLO en una sola fuente; cola persistente en el Collector; revisión mensual del presupuesto de error	Salto métrica → traza desde el panel; E1 detectado por una alerta	3,9
3	Que funcione sin empujar	Caos en CI semanal; muestreo adaptativo; control de cobertura de instrumentación; rotación de guardia y plantilla de postmortem	Una ejecución de caos fallida bloquea el merge; un servicio sin instrumentar falla la validación	4,3

Cada acción del mes 2 responde a un hallazgo concreto del laboratorio, no a una buena práctica genérica. El SLO vive hoy en tres sitios —el panel de Grafana, el detector de anomalías y las variables de Terraform— y nada impide que diverjan; E1 no se detectó en 120 segundos porque la degradación no cruzaba ningún umbral configurado, y ese dato existe desde el Game Day sin haber producido todavía una alerta nueva.

8. Despliegue en las dos nubes

	GCP — Cloud Run	GCP — GKE + Istio	AWS — ECS Fargate
Collector	Sidecar por revisión	Deployment único	Sidecar por task
Identidad entre servicios	Token OIDC del metadata server	Certificado mTLS de Istio	Security group + service discovery
Resiliencia	Código del cliente HTTP	Manifiestos de Istio	Código del cliente HTTP
Destino de las señales	Cloud Trace · Managed Prometheus · Cloud Logging		X-Ray · CloudWatch
Coste en reposo	≈ 0 (escala a cero)	≈ 0,20 USD/h	≈ 0,19 USD/h
Estado	**Desplegado y verificado**	Definido como código	**Desplegado y verificado**

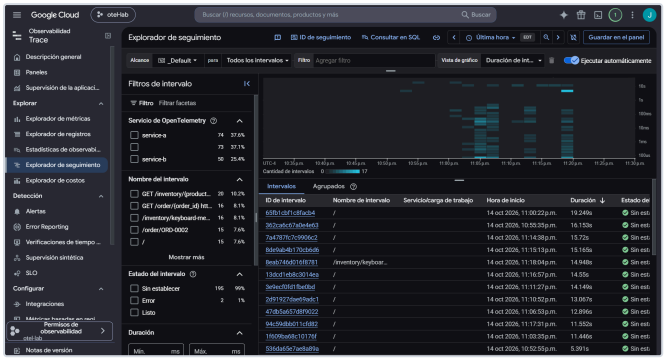
Los dos despliegues serverless están **vivos y comprobados de extremo a extremo**: una petición atraviesa los tres servicios y las tres señales comparten `trace_id` en ambas nubes. El mapa de servicios que compone AWS a partir de las trazas recibidas refleja exactamente la arquitectura diseñada, sin que nadie se la haya descrito:

Nodo	Tipo	Peticiones correctas	Errores
`service-a`	Contenedor gestionado	7	0
`service-b`	Contenedor gestionado	6	0
`data-service`	Contenedor gestionado	6	0
`appdb`	Database::SQL	18	0

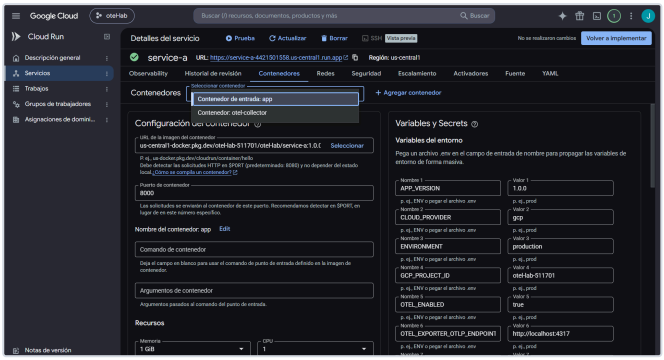
Las 18 consultas a base de datos frente a 19 peticiones HTTP son la huella de la arquitectura: cada pedido dispara una lectura en `service-a`, otra en `service-b` y la agregación analítica en `data-service`.

La misma imagen corre en los tres. `service-a` detecta el entorno por el esquema de la URL del destino: si es `https://` pide un token de identidad al metadata server, porque está hablando con un servicio de Cloud Run protegido por IAM; si es `http://` no lo pide, porque dentro de la malla la identidad la establece el certificado y no la cabecera.

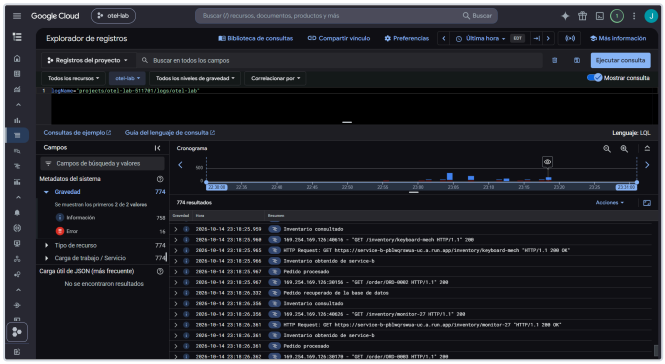
Toda la infraestructura es Terraform, repartida en cuatro raíces independientes por ciclo de vida: `gcp/` y `aws/` para los despliegues serverless, `gke/` para el clúster —que se levanta y se destruye el mismo día— y `observabilidad/` para alertas y paneles, que describen cómo se observa el sistema y deben sobrevivir a cualquier despliegue concreto.



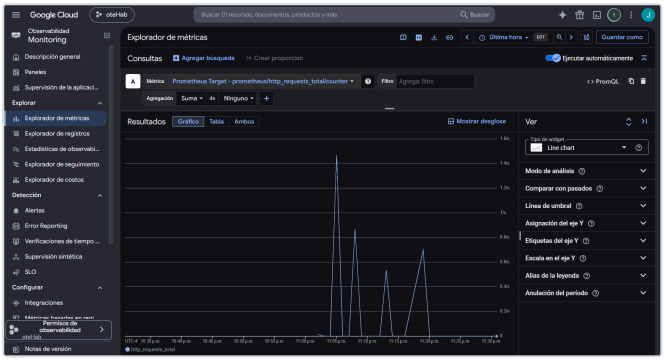
GCP **Cloud Trace** — Las mismas trazas tras cambiar sólo la configuración del Collector.



GCP **Cloud Run con sidecar** — El Collector viaja junto a la aplicación en cada revisión.



GCP **Cloud Logging** — Logs estructurados con `trace_id` como campo indexado.



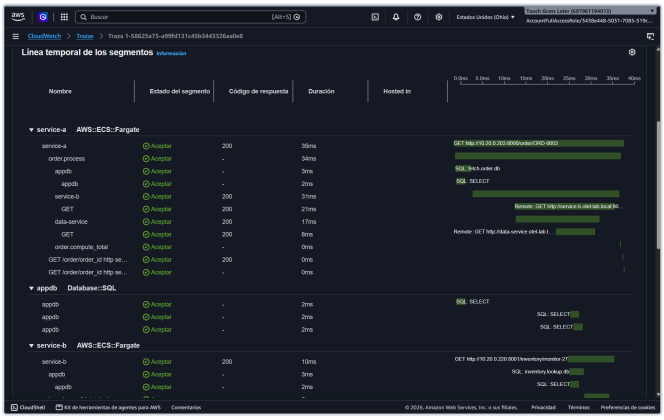
GCP **Managed Prometheus** — Las métricas de la aplicación, sin servidor que mantener.

El mismo pipeline en la otra nube

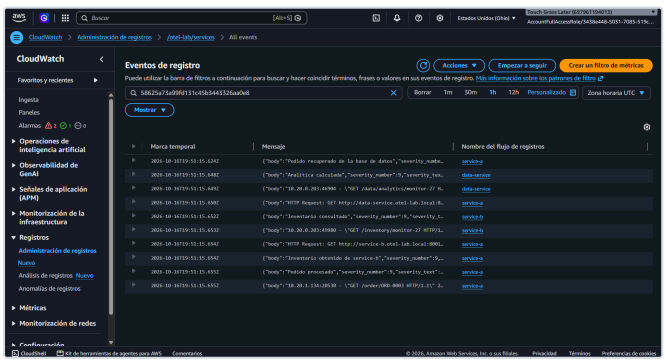
Las capturas siguientes son del segundo proveedor y muestran exactamente las mismas capacidades sobre herramientas distintas. No hubo cambios de codigo entre un despliegue y otro: solo una configuracion distinta del Collector.



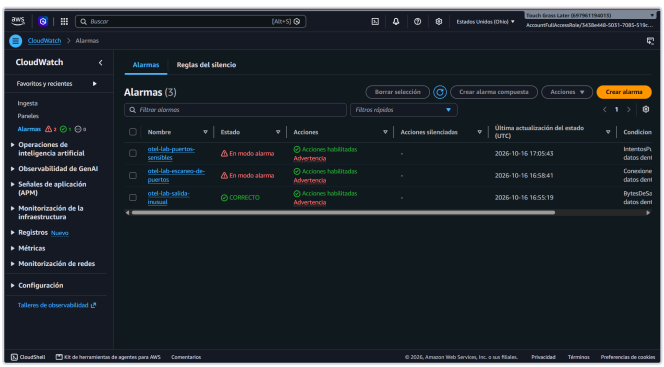
AWS **Mapa de servicios reconstruido desde las trazas** — Nadie describio esta topologia: la plataforma la deduce de los `traceparent` que viajan entre servicios. Coincide con la arquitectura disenada, incluido el nodo de base de datos.



AWS **Una peticion, tres servicios y la base** — Cascada completa bajo un unico identificador: 35 ms en total, con `service-b` y `data-service` ejecutandose en paralelo y los `SELECT` anidados bajo cada uno.



AWS **Logs de los tres servicios filtrados por trace_id** — El mismo identificador recupera las diez lineas que produjo la peticion, emitidas por tres procesos distintos. Es la correlacion traza-log funcionando sin herramientas propietarias.



AWS **Deteccion de red sobre trafico real** — Dos alarmas en rojo que nadie provoco, por escaneos de fondo contra las IP publicas. La tercera, en verde, mide el volumen de salida y confirma que no hay fuga.

9. Cómo reproducir el laboratorio

El repositorio se entrega etiquetado como `v1.0`. La estructura separa lo que se observa, lo que observa y dónde corre:

```
otel-lab/
  services/
    common/telemetry.py       bootstrap unico de las tres senales
    common/datastore.py       dos backends intercambiables (postgres / memoria)
    service-a/  service-b/  data-service/
  otel-collector/
    collector-local.yaml      Jaeger + Prometheus + Loki
    collector-gcp.yaml        Cloud Run  (sidecar)
    collector-gke.yaml        Kubernetes (k8sattributes + metricas de Istio)
    collector-aws.yaml        ECS Fargate (X-Ray + CloudWatch)
  k8s/                        manifiestos de la malla; 50- es el caos, aparte
  terraform/
    gcp/  aws/                despliegues serverless
    gke/                      cluster; se levanta y se destruye el mismo dia
    observabilidad/           alertas y paneles (Modulos B y C)
  aiops/
    detector.py               deteccion con baseline movil y correlacion
    escenario.sh             escenario controlado con ruido e incidente
  chaos/
    comun.sh                  probes SOT / DuringChaos / EOT
    experimentos/e1..e4       un experimento por hipotesis
    generador_carga.py        generador concurrente de un solo proceso
  docs/                       informes, evidencias y guias de despliegue
```

Paso	Comando	Resultado esperado
Levantar el stack local	`docker compose up -d`	Diez contenedores sanos; Grafana en `:3000`
Verificar la correlación	`bash scripts/verify-pipeline.sh`	Las tres señales comparten `trace_id`
Escenario de anomalías	`./aiops/escenario.sh`	Un incidente detectado, ruido descartado
Experimento de caos	`bash chaos/experimentos/e4-errores-data-service.sh`	Veredicto con hipótesis y presupuesto de error
Desplegar la malla	`./scripts/desplegar-gke.sh`	Clúster, Istio y aplicación; traza de extremo a extremo
Alertas y paneles	`terraform -chdir=terraform/observabilidad apply`	Seis políticas y el panel de seguridad
Regenerar este informe	`python docs/generar_reporte_integrador.py`	HTML con las cifras de las últimas mediciones

> **Control de coste.** El clúster de GKE es el único recurso con coste por hora relevante (≈ 0,20 USD/h) y el script de despliegue lo recuerda al terminar. `terraform -chdir=terraform/gke destroy` lo elimina por completo, incluido el balanceador que crea el *ingress gateway* de Istio. En AWS, `use_nat_gateway` se mantiene en `false` por el mismo motivo: un NAT Gateway cuesta más que el resto del despliegue junto.

10. Conclusiones

La instrumentación común es lo que hace barato crecer. Añadir el tercer microservicio costó importar un módulo y declarar sus instrumentos. La alternativa —instrumentar cada servicio por su cuenta— habría multiplicado el trabajo por tres y garantizado que los tres divergieran.

La malla separa la resiliencia del código. Los timeouts, los reintentos y los *circuit breakers* que antes vivían dentro del cliente HTTP de Python ahora son manifiestos versionados. Cambiar un plazo dejó de ser un despliegue para convertirse en un `kubectl apply`, y el cambio queda registrado en el control de versiones en lugar de enterrado en un commit de aplicación.

Correlacionar señales reduce el ruido sin perder cobertura. El escenario controlado lo midió: 50 % menos avisos sin acción asociada y ningún incidente perdido. El factor decisivo no es el algoritmo sino la exigencia de que dos señales independientes se degraden a la vez y de forma sostenida.

Una alerta vale lo que vale su primer paso. Adjuntar el `trace_id` de una petición que realmente falló cambia el punto de partida de quien responde: en lugar de buscar un ejemplo, abre el que ya tiene. El criterio de selección importa tanto como el mecanismo —la traza debe haber fallado *para el cliente*, porque un error interno que el sistema absorbió no explica un incidente, explica por qué no lo hubo.

Desplegar de verdad encuentra lo que ningún entorno local encuentra. El esquema de la base de datos dependía de una particularidad de la imagen oficial de PostgreSQL: el contenedor ejecuta los scripts de inicialización al crear su volumen, pero una base gestionada no tiene ese mecanismo. En local nunca falló; en la nube, el primer pedido devolvió `relation "orders" does not exist`. La corrección —que el propio servicio garantice su esquema con un script idempotente— es lo que hace que la misma imagen sirva en los tres entornos sin pasos manuales. El mismo patrón se repitió con un exporter del Collector que existe en una distribución y no en otra.

Un experimento que sólo sabe confirmar no es un experimento. Los cuatro experimentos verifican el estado estable antes de inyectar, comprueban que el fallo ocurrió de verdad antes de evaluar la hipótesis, y abortan si el sistema ya está fuera de su SLO. E4 terminó en «parcialmente confirmada»: la disponibilidad aguantó, pero el p99 subió más de lo previsto. Ese matiz es más útil que un «confirmada» limpio, porque señala dónde mirar después.

El resultado global es una plataforma con una media de madurez de **3,4 sobre 5** y una hoja de ruta donde cada acción responde a una carencia medida, no a una lista de buenas prácticas. La brecha principal es conocida y acotada: hay capacidades completas como código que todavía no se han puesto bajo presión, y la madurez se mide por lo que el sistema demuestra cuando se la pones.

> **Reproducibilidad.** Todas las cifras de este informe se leen de los ficheros JSON que generan los propios experimentos (`chaos/resultados/` y `aiops/resultados/`). El informe se reconstruye con `python docs/generar_reporte_integrador.py`; si un resultado falta, la celda aparece marcada como «sin dato» en vez de transcribirse a mano.